

Data Analytics (DA)

Module-End -3

Module End Assignment:

“Analyzing E-Learning Platform Purchases using MySQL”

SECTION 1: DATABASE SETUP & DATA ENTRY

The screenshot shows the MySQL Workbench interface with a script editor containing the following SQL code:

```
4
5  -- SECTION 1: DATABASE SETUP & DATA ENTRY
6
7  • DROP DATABASE IF EXISTS elearning_platform;
8  CREATE DATABASE elearning_platform;
9  USE elearning_platform;
10
11  -- -----
```

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec

The screenshot shows the MySQL Workbench interface with a script editor containing the following SQL code:

```
10
11  -- -----
12  -- Table: learners
13  -- -----
14  • CREATE TABLE learners (
15      learner_id INT PRIMARY KEY AUTO_INCREMENT,
16      full_name VARCHAR(100) NOT NULL,
17      country VARCHAR(50) NOT NULL
18  );
19
20  -- -----
```

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name...	0 row(s) affected	0.219 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmppy
- btmppy1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

219

220

221

222

223

224

225

226

227

228

229

```
-- Table: courses
CREATE TABLE courses (
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_name VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NULL, country VARCHAR(50) NOT NULL)	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100) NOT NULL, category VARCHAR(50) NOT NULL, unit_price DECIMAL(10,2) NOT NULL)	0 row(s) affected	0.031 sec

Object Info Session

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmppy
- btmppy1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

31

32

33

34

35

36

37

38

39

40

41

42

43

```
-- Table: purchases
CREATE TABLE purchases (
  purchase_id INT PRIMARY KEY AUTO_INCREMENT,
  learner_id INT NOT NULL,
  course_id INT NOT NULL,
  quantity INT NOT NULL,
  purchase_date DATE NOT NULL,
  CONSTRAINT fk_purchases_learner FOREIGN KEY (learner_id) REFERENCES learners(learner_id),
  CONSTRAINT fk_purchases_course FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	21:49:09	DROP DATABASE IF EXISTS elearning_platform	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'elearning_platform'; database doesn't exist	0.063 sec
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NULL, country VARCHAR(50) NOT NULL)	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100) NOT NULL, category VARCHAR(50) NOT NULL, unit_price DECIMAL(10,2) NOT NULL)	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, learner_id INT NOT NULL, course_id INT NOT NULL, quantity INT NOT NULL, purchase_date DATE NOT NULL, CONSTRAINT fk_purchases_learner FOREIGN KEY (learner_id) REFERENCES learners(learner_id), CONSTRAINT fk_purchases_course FOREIGN KEY (course_id) REFERENCES courses(course_id))	0 row(s) affected	0.172 sec

Object Info Session

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmppy
- btmppy1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
 - Tables
 - Views
 - Stored Procedures
 - Functions

Administration Schemas

Information

No object selected

43

44

45

46

47

48

49

50

51

52

53

```
-- Sample Data: learners (5)
INSERT INTO learners (full_name, country) VALUES
  ('Alice Johnson', 'USA'),
  ('Rahul Mehta', 'India'),
  ('Sophia Rossi', 'Italy'),
  ('Kenji Sato', 'Japan'),
  ('Maria Garcia', 'USA');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
2	21:49:10	CREATE DATABASE elearning_platform	1 row(s) affected	0.032 sec
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NULL, country VARCHAR(50) NOT NULL)	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100) NOT NULL, category VARCHAR(50) NOT NULL, unit_price DECIMAL(10,2) NOT NULL)	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, learner_id INT NOT NULL, course_id INT NOT NULL, quantity INT NOT NULL, purchase_date DATE NOT NULL, CONSTRAINT fk_purchases_learner FOREIGN KEY (learner_id) REFERENCES learners(learner_id), CONSTRAINT fk_purchases_course FOREIGN KEY (course_id) REFERENCES courses(course_id))	0 row(s) affected	0.172 sec
7	21:54:41	INSERT INTO learners (full_name, country) VALUES ('Alice Johnson', 'USA'), ('Rahul Mehta', 'India'), ('Sophia Rossi', 'Italy'), ('Kenji Sato', 'Japan'), ('Maria Garcia', 'USA')	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.047 sec

Object Info Session

MySQL Workbench - Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS: Filter objects, btmpy, btmpy1, chatbot_db, employee, flask_practice, form_data, leave_management_system, parkingmanagementsystem, plsql, practice, Tables, Views, Stored Procedures, Functions, Administration, Schemas, Information

No object selected

```

52
53
54 -- Sample Data: courses (6, across 5 categories; one course is never purchased)
55
56 INSERT INTO courses (course_name, category, unit_price) VALUES
57 ('Python for Beginners', 'Beginner', 999.00),
58 ('Advanced Data Science', 'Data Science', 4999.00),
59 ('Digital Marketing Basics', 'Marketing', 2499.00),
60 ('Full Stack Web Development', 'Web Development', 5999.00),
61 ('Excel Fundamentals', 'Beginner', 799.00),
62 ('UX Design Basics', 'Design', 3499.00);

```

Output: Action Output

#	Time	Action	Message	Duration / Fetch
3	21:49:10	USE elearning_platform	0 row(s) affected	0.000 sec
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_na...	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, cours...	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, l...	0 row(s) affected	0.172 sec
7	21:54:41	INSERT INTO learners (full_name, country) VALUES ('Alice Johnson', 'USA'), ('Rahul M...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.047 sec
8	21:55:28	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Beginner...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.031 sec

MySQL Workbench - Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS: Filter objects, btmpy, btmpy1, chatbot_db, employee, flask_practice, form_data, leave_management_system, parkingmanagementsystem, plsql, practice, Tables, Views, Stored Procedures, Functions, Administration, Schemas, Information

No object selected

```

64
65 -- Sample Data: purchases (8)
66
67 INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES
68 (1, 1, 3, '2026-01-05'), -- Alice - Python for Beginners
69 (1, 4, 1, '2026-02-10'), -- Alice - Full Stack Web Development
70 (2, 2, 2, '2026-01-15'), -- Rahul - Advanced Data Science
71 (2, 3, 2, '2026-03-01'), -- Rahul - Digital Marketing Basics
72 (3, 1, 1, '2026-01-20'), -- Sophia - Python for Beginners
73 (4, 4, 2, '2026-02-25'), -- Kenji - Full Stack Web Development
74 (4, 2, 1, '2026-03-10'), -- Kenji - Advanced Data Science
75 (5, 3, 3, '2026-01-30'), -- Maria - Excel Fundamentals
76 -- Note: course_id 6 (UX Design Basics) is intentionally never purchased

```

Output: Action Output

#	Time	Action	Message	Duration / Fetch
4	21:52:43	CREATE TABLE learners (learner_id INT PRIMARY KEY AUTO_INCREMENT, full_name VARCHAR(100) NOT NUL...	0 row(s) affected	0.219 sec
5	21:53:15	CREATE TABLE courses (course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100) NOT ...	0 row(s) affected	0.031 sec
6	21:53:50	CREATE TABLE purchases (purchase_id INT PRIMARY KEY AUTO_INCREMENT, learner_id INT NOT NULL, c...	0 row(s) affected	0.172 sec
7	21:54:41	INSERT INTO learners (full_name, country) VALUES ('Alice Johnson', 'USA'), ('Rahul Mehta', 'India'), ('Sophia Ross', ...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.047 sec
8	21:55:28	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Beginners', 'Beginner', 999.00), ...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.031 sec
9	21:56:21	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 3, '2026-01-05'), -- Alice - Pytho...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.015 sec

SECTION 2: DATA EXPLORATION USING JOINS

MySQL Workbench - Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator: elearning_analysis

SCHEMAS: Filter objects, btmpy, btmpy1, chatbot_db, employee, flask_practice, form_data, leave_management_system, parkingmanagementsystem, plsql, practice, Tables, Views, Stored Procedures, Functions, Administration, Schemas, Information

No object selected

```

79
80 -- SECTION 2: DATA EXPLORATION USING JOINS
81
82
83 -- INNER JOIN: only purchases that have a matching learner and course
84
85 SELECT
86     l.full_name AS learner_name,
87     c.course_name AS course_name,
88     c.category AS category,
89     p.quantity AS quantity,
90     ROUND(p.quantity * c.unit_price, 2) AS total_amount,
91     p.purchase_date AS purchase_date
92 FROM purchases p
93 INNER JOIN learners l ON (p.learner_id = l.learner_id)
94 INNER JOIN courses c ON (p.course_id = c.course_id)
95 ORDER BY (p.quantity * c.unit_price) DESC;

```

Result Grid

learner_name	course_name	category	quantity	total_amount	purchase_date
Kenji Sato	Full Stack Web Development	Web Development	2	11,998.00	2026-02-25
Rahul Mehta	Advanced Data Science	Data Science	2	9,998.00	2026-01-15
Alice Johnson	Full Stack Web Development	Web Development	1	5,999.00	2026-02-10
Kenji Sato	Advanced Data Science	Data Science	1	4,999.00	2026-03-10
Rahul Mehta	Digital Marketing Basics	Marketing	2	4,998.00	2026-03-01

Result 1 x

Output: Action Output

#	Time	Action	Message	Duration / Fetch
8	21:55:28	INSERT INTO courses (course_name, category, unit_price) VALUES ('Python for Beginners', 'Beginner', 999.00), ...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.031 sec
9	21:56:21	INSERT INTO purchases (learner_id, course_id, quantity, purchase_date) VALUES (1, 1, 3, '2026-01-05'), -- Alice - Pytho...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.015 sec
10	21:57:10	SELECT l.full_name AS learner_name, c.course_name AS course_name, c.category AS category, p.quantity AS quantity, ...	8 row(s) returned	0.078 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SHEMAS

Filter objects

btmysql
btmysql1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions

Administration Schemas

Information

No object selected

Query Editor

```
-- LEFT JOIN: all learners, including any with no purchases (none in this sample, but query is NULL-safe)
SELECT
  l.full_name AS learner_name,
  c.course_name AS course_name,
  c.category AS category,
  p.quantity AS quantity,
  FORMAT(IFNULL(p.quantity * c.unit_price, 0), 2) AS total_amount,
  p.purchase_date AS purchase_date
FROM learners l
LEFT JOIN purchases p ON l.learner_id = p.learner_id
LEFT JOIN courses c ON p.course_id = c.course_id
ORDER BY (p.quantity * c.unit_price) DESC
```

Result Grid

learner_name	course_name	category	quantity	total_amount	purchase_date
Kerji Sato	Full Stack Web Development	Web Development	2	11,998.00	2026-02-25
Rahul Mehta	Advanced Data Science	Data Science	2	9,998.00	2026-01-15
Alice Johnson	Full Stack Web Development	Web Development	1	5,999.00	2026-02-10
Kerji Sato	Advanced Data Science	Data Science	1	4,999.00	2026-03-10
Rahul Mehta	Digital Marketing Basics	Marketing	2	4,998.00	2026-03-01

Result 2

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:00:38	SELECT l.full_name AS learner_name, c.course_name AS course_name	AS course_na... 8 row(s) returned	0.016 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SHEMAS

Filter objects

btmysql
btmysql1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions

Administration Schemas

Information

No object selected

Query Editor

```
-- RIGHT JOIN: all courses, including any never purchased (surfaces UX Design Basics)
SELECT
  l.full_name AS learner_name,
  c.course_name AS course_name,
  c.category AS category,
  p.quantity AS quantity,
  FORMAT(IFNULL(p.quantity * c.unit_price, 0), 2) AS total_amount,
  p.purchase_date AS purchase_date
FROM purchases p
RIGHT JOIN courses c ON p.course_id = c.course_id
LEFT JOIN learners l ON p.learner_id = l.learner_id
ORDER BY (p.quantity * c.unit_price) DESC
```

Result Grid

learner_name	course_name	category	quantity	total_amount	purchase_date
Kerji Sato	Full Stack Web Development	Web Development	2	11,998.00	2026-02-25
Rahul Mehta	Advanced Data Science	Data Science	2	9,998.00	2026-01-15
Alice Johnson	Full Stack Web Development	Web Development	1	5,999.00	2026-02-10
Kerji Sato	Advanced Data Science	Data Science	1	4,999.00	2026-03-10
Rahul Mehta	Digital Marketing Basics	Marketing	2	4,998.00	2026-03-01

Result 3

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:00:38	SELECT l.full_name AS learner_name, c.course_name AS course_name	AS course_na... 8 row(s) returned	0.016 sec / 0.000 sec
2	22:01:26	SELECT l.full_name AS learner_name, c.course_name AS course_name	AS course_na... 9 row(s) returned	0.000 sec / 0.000 sec

SECTION 3: CORE ANALYTICAL QUERIES

Q1. Each learner's total spending with their country

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SHEMAS

Filter objects

btmysql
btmysql1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions

Administration Schemas

Information

No object selected

Query Editor

```
-- SECTION 3: CORE ANALYTICAL QUERIES
-- Q1. Each learner's total spending with their country
SELECT
  l.full_name AS learner_name,
  l.country AS country,
  FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spending
FROM learners l
JOIN purchases p ON l.learner_id = p.learner_id
JOIN courses c ON p.course_id = c.course_id
GROUP BY l.learner_id, l.full_name, l.country
ORDER BY SUM(p.quantity * c.unit_price) DESC
```

Result Grid

learner_name	country	total_spending
Kerji Sato	Japan	16,997.00
Rahul Mehta	India	14,996.00
Alice Johnson	USA	8,996.00
Maria Garcia	USA	2,997.00
Sophia Rossi	Italy	999.00

Result 4

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:00:38	SELECT l.full_name AS learner_name, c.course_name AS course_name	AS course_na... 8 row(s) returned	0.016 sec / 0.000 sec
2	22:01:26	SELECT l.full_name AS learner_name, c.course_name AS course_name	AS course_na... 9 row(s) returned	0.000 sec / 0.000 sec
3	22:03:43	SELECT l.full_name AS learner_name, l.country AS country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spending	AS country, FORMAT(SUM(... 5 row(s) returned	0.016 sec / 0.000 sec

Q2. Top 3 most purchased courses by quantity

The screenshot shows the MySQL Workbench interface with a query executed in the 'elearning_analysis' database. The query is as follows:

```
-- Q2. Top 3 most purchased courses by quantity
SELECT
  c.course_name AS course_name,
  c.category AS category,
  SUM(p.quantity) AS total_quantity_sold
FROM courses c
JOIN purchases p ON c.course_id = p.course_id
GROUP BY c.course_id, c.course_name, c.category
ORDER BY total_quantity_sold DESC, c.course_name ASC
LIMIT 3;
```

The result grid displays the following data:

course_name	category	total_quantity_sold
Python for Beginners	Beginner	4
Advanced Data Science	Data Science	3
Excel Fundamentals	Beginner	3

The output pane shows the execution details, including the time taken (22:04:32) and the number of rows returned (3 row(s) returned).

Q3. Each category's total revenue and number of unique learners

The screenshot shows the MySQL Workbench interface with a query executed in the 'elearning_analysis' database. The query is as follows:

```
-- Q3. Each category's total revenue and number of unique learners
SELECT
  c.category AS category,
  FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_revenue,
  COUNT(DISTINCT p.learner_id) AS unique_learners
FROM courses c
JOIN purchases p ON c.course_id = p.course_id
GROUP BY c.category
ORDER BY SUM(p.quantity * c.unit_price) DESC;
```

The result grid displays the following data:

category	total_revenue	unique_learners
Web Development	17,997.00	2
Data Science	14,997.00	2
Beginner	6,393.00	3
Marketing	4,998.00	1

The output pane shows the execution details, including the time taken (22:04:32) and the number of rows returned (3 row(s) returned).

Q4. Learners who purchased from more than one category

The screenshot shows the MySQL Workbench interface with a query executed in the 'elearning_analysis' database. The query is as follows:

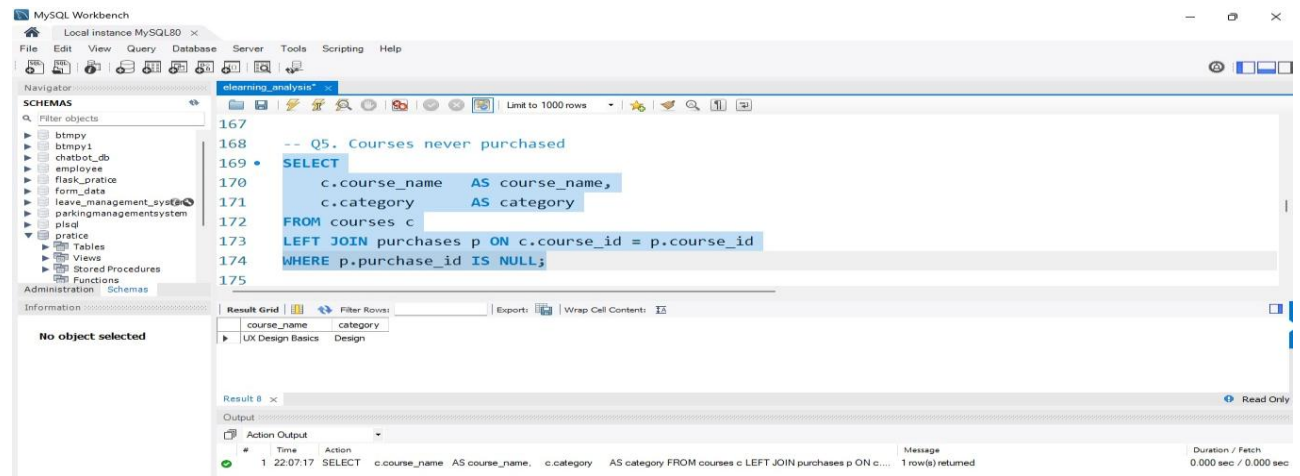
```
-- Q4. Learners who purchased from more than one category
SELECT
  l.full_name AS learner_name,
  COUNT(DISTINCT c.category) AS categories_purchased
FROM learners l
JOIN purchases p ON l.learner_id = p.learner_id
JOIN courses c ON p.course_id = c.course_id
GROUP BY l.learner_id, l.full_name
HAVING COUNT(DISTINCT c.category) > 1
ORDER BY categories_purchased DESC;
```

The result grid displays the following data:

learner_name	categories_purchased
Alice Johnson	2
Rahul Mehta	2
Kenji Sato	2

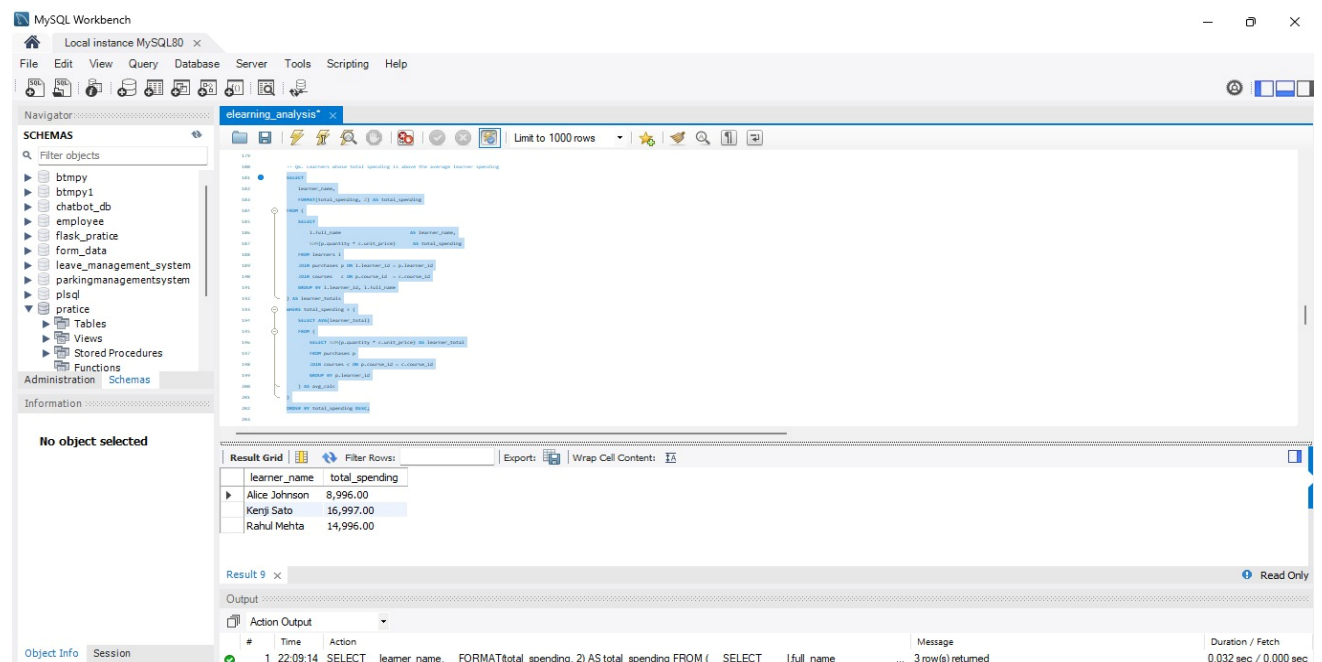
The output pane shows the execution details, including the time taken (22:05:30) and the number of rows returned (3 row(s) returned).

Q5. Courses never purchased



SECTION 4: SUBQUERIES & CORRELATED SUBQUERIES (Q6-Q8)

Q6. Learners whose total spending is above the average learner spending



Q7. Courses whose price is higher than any course in the 'Beginner' category

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

btmppy
btmppy1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions
Administration
Schemas
Information

No object selected

203
204
205
206
207
208
209
210
211
212
213
214

```
-- Q7. Courses whose price is higher than any course in the 'Beginner' category
SELECT
  course_name,
  category,
  FORMAT(unit_price, 2) AS unit_price
FROM courses
WHERE unit_price > ANY (
  SELECT unit_price FROM courses WHERE category = 'Beginner'
)
ORDER BY unit_price DESC;
```

Result Grid

course_name	category	unit_price
Python for Beginners	Beginner	999.00
Full Stack Web Development	Web Development	5,999.00
Advanced Data Science	Data Science	4,999.00
LUX Design Basics	Design	3,499.00
Digital Marketing Basics	Marketing	2,499.00

Result 10

Output

Action Output

Object Info Session

1 22:10:56 SELECT course_name, category, FORMAT(unit_price, 2) AS unit_price FROM courses WHERE unit_price > ANY (... 5 row(s) returned 0.016 sec / 0.000 sec

Q8. Learners who spent more than the average spending in their own country (correlated subquery)

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

btmppy
btmppy1
chatbot_db
employee
flask_practice
form_data
leave_management_system
parkingmanagementsystem
plsql
practise
Tables
Views
Stored Procedures
Functions
Administration
Schemas
Information

No object selected

215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```
-- Q8. Learners who spent more than the average spending in their own country (correlated subquery)
SELECT
  l.learner_name AS learner_name,
  l.country AS country,
  FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spending
FROM learners l
JOIN purchases p ON l.learner_id = p.learner_id
JOIN courses c ON p.course_id = c.course_id
GROUP BY l.learner_id, l.full_name, l.country
HAVING SUM(p.quantity * c.unit_price) > (
  SELECT AVG(country_total_spending)
  FROM (
    SELECT l.learner_id, l.country AS learner_country, SUM(p.quantity * c.unit_price) AS spend
    FROM learners l
    JOIN purchases p2 ON l.learner_id = p2.learner_id
    JOIN courses c2 ON p2.course_id = c2.course_id
    GROUP BY l.learner_id, l.country
  ) AS country_totals
  WHERE country_total_spending = l.country
)
ORDER BY total_spending DESC;
```

Result Grid

learner_name	country	total_spending
Alice Johnson	USA	8,996.00

Result 11

Output

Action Output

Object Info Session

1 22:13:09 SELECT l.full_name AS learner_name, l.country AS country, FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spending FROM learners l JOIN purchases p ON l.learner_id = p.learner_id JOIN courses c ON p.course_id = c.course_id GROUP BY l.learner_id, l.full_name, l.country HAVING SUM(p.quantity * c.unit_price) > (SELECT AVG(country_total_spending) FROM (SELECT l.learner_id, l.country AS learner_country, SUM(p.quantity * c.unit_price) AS spend FROM learners l JOIN purchases p2 ON l.learner_id = p2.learner_id JOIN courses c2 ON p2.course_id = c2.course_id GROUP BY l.learner_id, l.country) AS country_totals WHERE country_total_spending = l.country) ORDER BY total_spending DESC; 1 row(s) returned 0.000 sec / 0.000 sec

SECTION 5: CTE, CASE, VIEW, AND NULL HANDLING (Q9-Q12)

Q9. CTE: total spending per learner, then filter learners spending above 10,000

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmppy
- btmppy1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
- Tables
- Views
- Stored Procedures
- Functions
- Schemas

Information

No object selected

Query Editor

```

227 -- SECTION 10: CTEs, CASE, VIEWs, AND NULL HANDLING (Q9 Q12)
228 --
229 -- Q9: CTE: total spending per learner, then filter learners spending above 10,000
230
231 WITH learner_spending AS (
232     SELECT
233         l.learner_id,
234         l.full_name AS learner_name,
235         SUM(p.quantity * c.unit_price) AS total_spending
236     FROM learners l
237     JOIN purchases p ON l.learner_id = p.learner_id
238     JOIN courses c ON p.course_id = c.course_id
239     GROUP BY l.learner_id, l.full_name
240 )
241
242 SELECT
243     learner_name,
244     FORMAT(total_spending, 2) AS total_spending
245 FROM learner_spending
246 WHERE total_spending > 10000
247 ORDER BY total_spending DESC;

```

Result Grid

learner_name	total_spending
Kenji Sato	16,997.00
Rahul Mehta	14,996.00

Result 12 x

Output

Action Output

Object Info Session

1 22:14:09 WITH learner_spending AS (SELECT l.learner_id, l.full_name AS learner_name, SUM(p.quantity * c.unit_price... 2 row(s) returned 0.000 sec / 0.000 sec

Q10. CASE: classify learners by spending tier

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmppy
- btmppy1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
- Tables
- Views
- Stored Procedures
- Functions
- Schemas

Information

No object selected

Query Editor

```

259 -- Q10. CASE: classify learners by spending tier
260
261 WITH learner_spending AS (
262     SELECT
263         learner_name,
264         FORMAT(total_spending, 2) AS total_spending,
265         CASE
266             WHEN total_spending > 15000 THEN 'High Value'
267             WHEN total_spending BETWEEN 8000 AND 15000 THEN 'Medium Value'
268             ELSE 'Low Value'
269         END AS spending_tier
270     FROM learner_spending
271 )
272
273 SELECT
274     learner_name,
275     total_spending,
276     spending_tier
277 FROM learner_spending
278 ORDER BY total_spending DESC;

```

Result Grid

learner_name	total_spending	spending_tier
Sophia Rossi	999.00	Low Value
Alice Johnson	8,996.00	Medium Value
Maria Garcia	2,997.00	Low Value
Kenji Sato	16,997.00	High Value
Rahul Mehta	14,996.00	Medium Value

Result 13 x

Output

Action Output

Object Info Session

1 22:16:24 WITH learner_spending AS (SELECT l.full_name AS learner_name, SUM(p.quantity * c.unit_price... 5 row(s) returned 0.016 sec / 0.000 sec

Q11. NULL Handling: all courses, replacing NULL purchase counts with 0

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- btmppy
- btmppy1
- chatbot_db
- employee
- flask_practice
- form_data
- leave_management_system
- parkingmanagementsystem
- plsql
- practice
- Tables
- Views
- Stored Procedures
- Functions
- Schemas

Information

No object selected

Query Editor

```

279
280 -- Q11. NULL Handling: all courses, replacing NULL purchase counts with 0
281
282 SELECT
283     c.course_name AS course_name,
284     c.category AS category,
285     IFNULL(COUNT(p.purchase_id), 0) AS purchase_count,
286     COALESCE(SUM(p.quantity), 0) AS total_quantity_sold
287 FROM courses c
288 LEFT JOIN purchases p ON c.course_id = p.course_id
289 GROUP BY c.course_id, c.course_name, c.category
290 ORDER BY purchase_count DESC;

```

Result Grid

course_name	category	purchase_count	total_quantity_sold
Python for Beginners	Beginner	2	4
Advanced Data Science	Data Science	2	3
Full Stack Web Development	Web Development	2	3
Digital Marketing Basics	Marketing	1	2
Excel Fundamentals	Beginner	1	3

Result 15 x

Output

Action Output

Object Info Session

1 22:17:29 SELECT c.course_name AS course_name, c.category AS category, IFNULL(COUNT(p.purchase_id), 0) AS purchase_count, COALESCE(SUM(p.quantity), 0) AS total_quantity_sold 6 row(s) returned 0.000 sec / 0.000 sec

Q12. VIEW: category_performance_view

The screenshot shows the MySQL Workbench interface with a query editor window titled 'elearning_analysis'. The query is as follows:

```
290
291 -- Q12. VIEW: category_performance_view
292 • CREATE OR REPLACE VIEW category_performance_view AS
293 SELECT
294     c.category AS category,
295     SUM(p.quantity * c.unit_price) AS total_revenue,
296     COUNT(p.purchase_id) AS number_of_purchases,
297     ROUND(SUM(p.quantity * c.unit_price) / COUNT(p.purchase_id), 2) AS avg_revenue_per_purchase
298 FROM courses c
299 JOIN purchases p ON c.course_id = p.course_id
300 GROUP BY c.category;
```

The output pane at the bottom shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	22:18:59	CREATE OR REPLACE VIEW category_performance_view AS SELECT c.category	AS c... 0 row(s) affected	0.016 sec

Query the view

The screenshot shows the MySQL Workbench interface with a query editor window titled 'elearning_analysis'. The query is as follows:

```
301
302 -- Query the view
303 • SELECT
304     category,
305     FORMAT(total_revenue, 2) AS total_revenue,
306     number_of_purchases,
307     FORMAT(avg_revenue_per_purchase, 2) AS avg_revenue_per_purchase
308 FROM category_performance_view
309 ORDER BY total_revenue DESC;
```

The output pane at the bottom shows the execution result in a table format:

category	total_revenue	number_of_purchases	avg_revenue_per_purchase
Beginner	6,393.00	3	2,131.00
Marketing	4,998.00	1	4,998.00
Web Development	17,997.00	2	8,998.50
Data Science	14,997.00	2	7,498.50

The output pane also shows the execution result:

#	Time	Action	Message	Duration / Fetch
1	22:19:56	SELECT category, FORMAT(total_revenue, 2) AS total_revenue, number_of_purchases, FORMAT(avg_reve...	4 row(s) returned	0.000 sec / 0.000 sec